

Object-oriented Programming with Python

SOMCHAI CHEINGPONGPAN

DEPARTMENT OF INFORMATION TECHNOLOGY

FACULTY OF INDUSTRIAL AND TECHNOLOGY MANAGEMENT

Object-oriented Programming with Python

2

- ▶ **Class and Object**
- ▶ **Principles of Object-oriented**
- ▶ **การสร้างและใช้งาน Class เบื้องต้น**
- ▶ **Constructor**
- ▶ **Class แบบ Encapsulation**
- ▶ **การใช้งาน Class แบบ Static**
- ▶ **Class แบบ Inheritance**
- ▶ **การเขียนโปรแกรม การทำงานแบบ Stack**

Class and Object

- ▶ **คลาส (Class)** คือ ต้นแบบของวัตถุ(blueprint of object) การสร้างวัตถุขึ้นมาอย่างหนึ่ง จะต้องสร้างคลาสขึ้นมาเป็นโครงสร้างต้นแบบสำหรับวัตถุก่อนเสมอ
- ▶ **วัตถุ (Object)** คือ วัตถุหรือสิ่งของที่มีอยู่จริงบนโลก เป็นได้ทั้งรูปธรรม และนามธรรม วัตถุแต่ละชิ้นสามารถกำหนดคุณสมบัติเฉพาะของตัวเองได้ ทำให้แต่ละวัตถุมีความแตกต่างกัน แต่คุณสมบัติพื้นฐาน ยังได้รับมาจากคลาส หรือแม่แบบเหมือนเดิม

Class and Object

4

คลาสในการเขียนโปรแกรม คือการรวบรวมกลุ่มของคำสั่งที่มีความสัมพันธ์กัน ซึ่งมีส่วนประกอบ 2 อย่างคือ

- ▶ คุณลักษณะ (Attribute หรือ Data) คือข้อมูลที่บอกคุณลักษณะทั่วไป หรือ คุณสมบัติเฉพาะตัวของวัตถุว่ามีข้อมูลอะไรบ้าง
- ▶ พฤติกรรม (Behavior หรือ Method) คือ สิ่งที่วัตถุสามารถกระทำออกมาได้ โดยเกี่ยวข้องกับข้อมูล เช่น การคำนวณ หาสังแสดงข้อมูล

โดยจะเรียก Attribute และ Behavior ว่าเป็น member ของคลาส

การเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming : OOP) คือ แนวคิดการเขียนโปรแกรม ที่มองทุกสิ่งทุกอย่างให้เป็นวัตถุ (object)

Principles of Object-oriented

5

- ▶ **Encapsulation**
- ▶ **Inheritance**
- ▶ **Polymorphism**

การสร้างและใช้งาน Class เบื้องต้น

6

การสร้าง Class ใน Python ใช้คำสั่ง `class` ตามด้วยชื่อ Class

▶ ตัวอย่างการสร้าง Class ใน Python:

```
class Dog:
```

```
    def __init__(self):
```

```
        self.name = 'Buddy'
```

```
        self.age = 3
```

▶ การสร้าง Object จาก Class ใช้คำสั่งดังนี้:

```
my_dog = Dog( ) # instance
```

การสร้างและใช้งาน Class เบื้องต้น

7

Oop_code >  Oop1_basic.py > ...

```
1  # สร้าง class
2  class Dog:
3      def __init__(self):
4          self.name = 'Buddy'
5          self.age = 1
6
7  # สร้าง Object
8  my_dog = Dog()
9  print(my_dog.name, my_dog.age)
10
```

Constructor เป็นเมธอดตัวหนึ่งภายในคลาส จะทำงานโดยอัตโนมัติทันทีหลังจากสร้าง **object** ในภาษาโปรแกรมอย่าง **Java** จะเป็นเมธอดที่มีชื่อเดียวกับชื่อคลาส แต่ในภาษา **Python** จะเป็นเมธอดพิเศษที่มีชื่อว่า **`__init__()`**

Constructor ใน **Python** มี 2 ชนิด คือ

- ▶ **Constructor แบบไม่มีพารามิเตอร์ (Non-Parameterized Constructor)**
- ▶ **Constructor แบบมีพารามิเตอร์ (Parameterized Constructor)**

Non-Parameterized Constructor

9

Oop_code >  Oop1_basic.py > ...

```
1  # สร้าง class
2  class Dog:
3      def __init__(self):
4          self.name = 'Buddy'
5          self.age = 1
6
7  # สร้าง Object
8  my_dog = Dog()
9  print(my_dog.name, my_dog.age)
10
```

Parameterized Constructor

10

Oop_code >  Oop1_parameter.py > ...

```
1  # สร้าง class
2  class Dog:
3      def __init__(self, name, age): # มี parameter
4          self.name = name
5          self.age = age
6
7  # สร้าง Object
8  my_dog = Dog('Buddy', 2)
9  print(my_dog.name, my_dog.age)
10
```

ข้อควรระวังในการกำหนด Constructor

ในภาษา Python ไม่แนะนำให้มี Constructor อยู่ในคลาสเดียวกันมากกว่า 1 ตัวเนื่องจาก Interpreter ของ Python จะเลือกทำงานใน Constructor ตัวหลังสุดและมีจำนวน parameter ที่มากกว่า

Parameterized Constructor

12

Oop_code >  Oop1_parameter.py >  Dog >  __init__

```
1  # สร้าง class
2  class Dog:
3      def __init__(self): # no parameter
4          self.name = 'name'
5          self.age = 1
6      def __init__(self, name, age): # มี parameter
7          self.name = name
8          self.age = age
9
10 # สร้าง Object
11 my_dog1 = Dog()
12 my_dog2 = Dog('Buddy', 2)
13 print(my_dog1.name, my_dog1.age)
14 print(my_dog2.name, my_dog2.age)
15
```

Parameterized Constructor

13

parameter ที่อยู่ใน constructor สามารถกำหนดเป็นค่า default ได้ และเมื่อมีการสร้าง object จะระบุค่าที่ส่งไปใน parameter หรือไม่ก็ได้

Oop_code >  Oop1_default.py >  Dog >  __init__

```
1  # สร้าง class
2  class Dog:
3      def __init__(self, name='Buddy', age=1): # มี default
4          self.name = name
5          self.age = age
6
7  # สร้าง Object
8  my_dog1 = Dog()
9  my_dog2 = Dog('Buddy', 2)
10 print(my_dog1.name, my_dog1.age)
11 print(my_dog2.name, my_dog2.age)
12
```

Class แบบ Encapsulation

14

การห่อหุ้ม (Encapsulation) คือคลาสที่ประกอบตัวแปรและเมธอด โดยมี การกำหนดสิทธิ์ในการเข้าถึงสมาชิกภายในคลาส ไม่ว่าจะมาจากภายในหรือ ภายนอกคลาสก็ตาม ทำให้ข้อมูลมีความปลอดภัย และเป็นความลับ

ระดับความสามารถในการเข้าถึงตัวแปร และเมธอดในคลาส มี 3 ระดับ คือ

- เข้าถึงตัวแปรและเมธอดได้ทั้งภายในคลาส และภายนอกคลาส
- เข้าถึงตัวแปรและเมธอดได้เฉพาะภายในคลาสตัวเอง และคลาสที่สืบทอดมา
- เข้าถึงตัวแปรและเมธอดได้เฉพาะภายในคลาสตัวเองเท่านั้น

Class แบบ Encapsulation

15

คีย์เวิร์ดที่ใช้ระบุระดับความสามารถในการเข้าถึงตัวแปร และเมธอดของคลาส
ในภาษา Java และ Python

ระดับสิทธิ์การเข้าถึง	ภาษา Java	ภาษา Python
เข้าถึงได้ทุกคลาส	public	ไม่มีคีย์เวิร์ด
เข้าถึงได้ในคลาสตัวเอง และคลาสที่สืบทอด	protected	_ (single underscore)
เข้าถึงได้เฉพาะในคลาสตัวเอง	private	__ (double underscore)

ตัวอย่าง Class แบบ Encapsulation

16

Oop_Encapsulation >  Oop1_encap.py > ...

Person 1 : Somchai, Cheingpongpan
Person 2 : Jason, Smith

```
1  # สร้าง class
2  class Person:
3      def __init__(self, name, surname):
4          self.__name = name
5          self.__surname = surname
6      def toString(self):
7          return f"{self.__name}, {self.__surname}"
8
9  # สร้าง Object
10 p1 = Person('Somchai', 'Cheingpongpan')
11 p2 = Person('Jason', 'Smith')
12 print('Person 1 : ', p1.toString())
13 print('Person 2 : ', p2.toString())
```


Encapsulation : private accessing

17

หลักการสำคัญอย่างหนึ่งของ Encapsulation คือ การกำหนดตัวแปรหรือเมธอดให้เป็น private เพื่อควบคุมไม่ให้มีการเข้าถึงข้อมูลแบบ public แต่ถ้าจะต้องการเข้าถึงตัวแปร หรือเมธอดที่เป็น private ในภาษา Python ก็สามารถทำได้หลายวิธี เช่น

- ▶ Underscore accessing
- ▶ Getter & Setter
- ▶ Property

Encapsulation : Underscore accessing

18

การเรียกชื่อ object เชื่อมด้วยจุด ตามด้วยเครื่องหมาย _ (underscore) หน้าชื่อคลาส ต่อด้วย __ (double underscore) หน้าชื่อตัวแปร หรือเมธอด

- ▶ ตัวแปร -> object_name._Class_name__attribute_name
- ▶ เมธอด -> object_name._Class_name__method_name()

ตัวอย่าง Encapsulation : Underscore accessing

19

Oop_Encapsulation >  Oop2_underscore_access.py > ...

Person 1 : Somchai, Cheingpongpan
New Person 1 : Jason, Smith

```
1  # สร้าง class
2  class Person:
3      def __init__(self, name, surname):
4          self.__name = name
5          self.__surname = surname
6      def toString(self):
7          return f"{self.__name}, {self.__surname}"
8
9  # สร้าง Object
10 p1 = Person('Somchai', 'Cheingpongpan')
11 print('Person 1 : ', p1.toString())
12 p1._Person__name = 'Jason'
13 p1._Person__surname = 'Smith'
14 print('New Person 1 : ', p1.toString())
15
```

ตัวอย่าง Encapsulation : Getter & Setter

20

ตัวอย่าง การใช้ Getter & Setter

Oop_Encapsulation >  Oop3_setget.py >  Person >  setSurname

```
1  # สร้าง class
2  class Person:
3      def __init__(self, name, surname):
4          self.__name = name
5          self.__surname = surname
6      def setName(self, name):
7          self.__name = name
8      def getName(self):
9          return self.__name
10     def getSurname(self):
11         return self.__surname
12     def setSurname(self, surname):
13         self.__surname = surname
14     def toString(self):
15         return f"{self.__name}, {self.__surname}"
16
```

ตัวอย่าง Encapsulation : Getter & Setter

21

```
17  # สร้าง Object
18  p1 = Person('Somchai', 'Cheingpongpan')
19  print('Person 1 : ', p1.toString())
20  p1.setName('Jason')
21  p1.setSurname('Smith')
22  print('New Person 1 : ', p1.getName(), ', ', p1.getSurname())
23
```

```
Person 1 : Somchai, Cheingpongpan
New Person 1 : Jason, Smith
```

Encapsulation : Property

22

property คือการสร้าง getter และ setter ในสไลด์ของภาษา Python ทำได้ 2 วิธี

- ▶ ใช้ฟังก์ชัน `property()` แล้วรับค่าพารามิเตอร์ที่เป็น getter และ setter
- ▶ แอด Decorator ชื่อว่า `@property` ครอบไว้ด้านบนเมธอดที่เป็น getter ส่วนเมธอดที่เป็น setter ให้แอด Decorator เป็นชื่อ `@method_name.setter` ครอบไว้บนเมธอดที่เป็น setter โดยชื่อเมธอดจะเหมือนกันทั้ง getter และ setter

ตัวอย่าง Encapsulation : Property 1

23

ตัวอย่าง การเข้าถึงตัวแปร __amount โดยใช้ฟังก์ชัน property()

Oop_Encapsulation > Opp4_property.py > Person

```
1  # สร้าง class
2  class Person:
3      def __init__(self, name, surname):
4          self.__name = name
5          self.__surname = surname
6      def setName(self, name):
7          self.__name = name
8      def getName(self):
9          return self.__name
10     def getSurname(self):
11         return self.__surname
12     def setSurname(self, surname):
13         self.__surname = surname
14     def toString(self):
15         return f"{self.__name}, {self.__surname}"
16
```

ตัวอย่าง Encapsulation : Property 1

24

การเข้าถึงตัวแปร __amount โดยใช้ฟังก์ชัน property()

```
17      # use property function
18      name = property(getName, setName)
19      surname = property( getSurname, setSurname)
20
21  # สร้าง Object
22  p1 = Person('Somchai', 'Cheingpongpan')
23  print('Person 1 : ' , p1.toString())
24  p1.name = 'Jason'
25  p1.surname = 'Smith'
26  print('New Person 1 : ', p1.name ,', ', p1.surname )
27
```


ตัวอย่าง Encapsulation : Property 2

25

▶ ตัวอย่าง การใช้ แอด Decorator property ในไพธอน

Oop_Encapsulation >  Oop5_property.py >  Person

```
1  # สร้าง class
2  class Person:
3      def __init__(self, name, surname):
4          self.__name = name
5          self.__surname = surname
6      @property
7      def name(self):
8          return self.__name
9      @name.setter
10     def name(self, name):
11         self.__name = name
12     @property
13     def surname(self):
14         return self.__surname
15     @surname.setter
16     def surname(self, surname):
17         self.__surname = surname
```

ตัวอย่าง Encapsulation : Property 2

26

- ▶ การใช้ แอด Decorator property ในไพธอน

```
18
19     def toString(self):
20         return f"{self.__name}, {self.__surname}"
21
22 # สร้าง Object
23 p1 = Person('Somchai', 'Cheingpongpan')
24 print('Person 1 : ', p1.toString())
25 p1.name = 'Jason'
26 p1.surname = 'Smith'
27 print('New Person 1 : ', p1.name, ', ', p1.surname )
28
```

Workshop 1 : Student.py

27

WS_oop >  student.py >  Student >  toGrade

```
1  class Student:
2      def __init__(self, id='', name='', score=0.0 ):
3          self.__id = id
4          self.__name = name
5          self.__score = score
6      @property
7      def id(self):
8          return self.__id
9      @id.setter
10     def id(self, id):
11         self.__id = id
12     @property
13     def name(self):
14         return self.__name
15     @name.setter
16     def name(self, name):
17         self.__name = name
```

Workshop 1 : Student.py

28

```
18     @property
19     def score(self):
20         I         return self.__score
21     @score.setter
22     def score(self, score):
23         self.__score = score
24
25     def getGrade(self):
26         GRADES = ['A', 'B+', 'B', 'C+', 'C', 'D+', 'D', 'F']
27         SCORES = [80, 75, 70, 65, 60, 55, 50, 0]
28         for n in range(len(GRADES)):
29             if self.__score >= SCORES[n]:
30                 return GRADES[n]
31     def toString(self):
32         return f'{self.__id, self.__name, self.__score}'
33     def toGrade(self):
34         return f'{self.toString() , self.getGrade()}'
35
```

Workshop 1 : WS_student.py

29

WS_oop >  Ws_Student.py > ...

```
1  from student import *
2
3  def addStudent():
4      std = Student()
5      std.id = input('Enter student id : ')
6      std.name = input('Enter student name : ')
7      std.score = int(input('Enter student score : '))
8      stds.append( std )
9
10 def reportStudent():
11     for std in stds:
12         print(std.toGrade())
13
14 # main program
15 menuStr = '    Main Menu\n=====
16 menuStr += ' 1. Add Student\n'
17 menuStr += ' 2. Report Grade\n 3. Exit\nEnter choice : '
```

Workshop 1 :

30

```
18  stds = []
19  choices = ['1', '2', '3']
20  while True:
21      print(menuStr, end='')
22      choice = input()
23      if choice in choices:
24          if choice == '1':
25              addStudent()
26              pass
27          elif choice == '2':
28              reportStudent()
29              pass
30          elif choice == '3':
31              break
32
33  print('Exit Program....')
```

Static เป็นวิธีการเรียกใช้งานตัวแปร หรือเมธอด โดยไม่ต้องสร้าง **object** ซึ่งจะใช้งานหน่วยความจำเฉพาะการทำงานในครั้งแรกเท่านั้น เมื่อจบการทำงานแล้ว จะคืนหน่วยความจำทันที และการใช้งานครั้งต่อไป จะใช้ค่าจากหน่วยความจำเดิม

ควรใช้ **static** ในกรณีดังต่อไปนี้

- เป็นค่าคงที่ หรือไม่ต้องการเปลี่ยนแปลงค่า
- นำค่าตัวแปร หรือเมธอดไปใช้กับหลายๆ คลาส

การใช้งาน Class แบบ Static

32

ในภาษา Python แอตทริบิวต์ (ตัวแปร) ในคลาส จะเป็น static อยู่แล้ว สามารถเรียกใช้ผ่าน "ชื่อคลาส.ชื่อตัวแปร" ได้เลย แต่สำหรับเมธอดนั้น ถ้าต้องการกำหนดให้เมธอดใด เป็น static ไม่ต้องใส่ self ไว้ในพารามิเตอร์ แต่สามารถรับค่าพารามิเตอร์รูปแบบอื่นๆ ได้ตามปกติ

การเรียกใช้งานตัวแปร หรือเมธอด non-static และ static ภายในคลาส

เรียกใช้งานตัวแปร -> `class_name.attribute_name`

เรียกใช้งานฟังก์ชัน -> `class_name.method_name();`

สรุปการเรียกใช้งานเมธอด non-static และ static กับคลาส

การเรียกใช้งาน เมธอด	ภายในคลาส	ภายนอกคลาส
ไม่เป็น static	ต้องสร้าง object และเรียกใช้งานผ่าน object	ต้องสร้าง object และเรียกใช้งานผ่าน object
เป็น static	เรียกชื่อคลาส.ชื่อเมธอด	เรียกชื่อคลาส.ชื่อเมธอด

ตัวอย่างการสร้างและใช้งาน Static

34

WS_oop > Employee.py > Employee > getSalary

```
1  class Employee:
2      count = 0      # static variable
3      def __init__(self, name, salary):
4          self.empName = name
5          self.empSalary = salary
6          Employee.count += 1
7      def setName(self, name):
8          self.empName = name
9      def getName(self):
10         return self.empName
11     def setSalary(self, salary):
12         self.empSalary = salary
13     def getSalary(self):
14         return self.empSalary
15     def toString(self):
16         return f'{self.empName} : {str(self.empSalary)}'
```

ตัวอย่างการสร้างและใช้งาน Static

35

```
17     @staticmethod
18     def getCount():          # static method
19         return Employee.count
20
21 if __name__ == "__main__":
22     print('Object of Employee = ', Employee.getCount())
23     emp1 = Employee('Somchai', 26800.00)
24     emp2 = Employee('John', 21800.00)
25     print('Employee 1 : ', emp1.toString())
26     print('Employee 2: ', emp2.toString())
27     print('Now Object of Employee = ', Employee.getCount())
28
```

Class แบบ Inheritance

36

การสืบทอด (Inheritance) คือการที่คลาสลูก(sub class)รับตัวแปรและเมธอดมาจากคลาสแม่ (super class) คลาสลูกที่สืบทอดมาจากคลาสแม่ จะมีความสามารถ 2 อย่าง

- ▶ ใช้งานตัวแปร และเมธอดจากคลาสแม่ได้
- ▶ เพิ่มตัวแปร และเมธอดในคลาสตัวเองได้

วิธีการสืบทอดคลาสในภาษา Python จะใช้ชื่อคลาสแม่ ไปอยู่ในวงเล็บต่อท้ายชื่อคลาสลูก และสามารถสืบทอดได้มากกว่า 1 คลาสในเวลาเดียวกัน

Inheritance: การใช้ inheritance ในไพธอน

37

▶ ตัวอย่าง: Class Person กับ Class Student

Oop4_Inheritance > Exam_inherit.py > ...

```
1 class Person:
2     def __init__(self, name):
3         self.__name = name
4     def setName(self, name):
5         self.__name = name
6     def getName(self):
7         return self.__name
8
```

```
9 class Student(Person):
10     def __init__(self, name, gpa):
11         super().__init__(name)
12         self.__gpa = gpa
13     def setGpa(self, gpa):
14         self.__gpa = gpa
15     def getGpa(self):
16         return self.__gpa
17
```

Inheritance: การใช้ inheritance ในไพธอน

38

▶ ตัวอย่าง: Class Person กับ Class Student

```
18  # main I
19  if __name__ == "__main__":
20      std = Student('somchai', 3.10)
21      print('Data student :')
22      print('Name :', std.getName() )
23      print('Gpa :', std.getGpa() )
24
```

Inheritance : (method overriding)

39

override คือการทำให้เมธอดของคลาสลูก (sub class) ทำงานแตกต่างจากเมธอด ของคลาสแม่ (super class) โดยจะมีรูปแบบของเมธอดที่เหมือนกัน ซึ่งเรียกแบบง่ายๆ ว่า "การเขียนทับ" นั่นเอง ให้ใช้คำสั่ง `super().method_name()`

Oop4_Inheritance > Exam_inherit_override.py > ...

```
1 class Person:
2     def __init__(self, name):
3         self.__name = name
4     def setName(self, name):
5         self.__name = name
6     def getName(self):
7         return self.__name
8     def toString(self):
9         return 'Name : ' + self.getName()
10
```

Inheritance : (method overriding)

40

```
11 class Student(Person):
12     def __init__(self, name, gpa):
13         super().__init__(name)
14         self.__gpa = gpa
15     def setGpa(self, gpa):
16         self.__gpa = gpa
17     def getGpa(self):
18         return self.__gpa
19     def toString(self):
20         return super().toString() + '\nGpa : ' + str(self.getGpa())
21
22 # main
23 if __name__ == "__main__":
24     std = Student('somchai', 3.10)
25     print('Data student :\n'+ std.toString())
26
```


การเขียนโปรแกรม การทำงานแบบ Stack

41

Stack (สแตก) เป็นโครงสร้างข้อมูลแบบหนึ่งทำงานตามหลักการ **LIFO (Last In, First Out)** หรือ “เข้าหลังออกก่อน” หมายความว่า ข้อมูลที่ถูกเพิ่มเข้ามาล่าสุดจะถูกนำออกก่อนเสมอ

หลักการทำงานของ Stack Stack มีการทำงานหลัก ๆ อยู่ 2 อย่างคือ:

1. **Push** – การเพิ่มข้อมูลเข้าไปใน Stack
2. **Pop** – การนำข้อมูลออกจาก Stack (เอาตัวบนสุดออก)

นอกจากนี้ยังมีฟังก์ชันเสริมที่มักใช้ร่วมกัน เช่น:

- **size ()** – ตรวจสอบขนาดของ Stack
- **peek (หรือ Top)** – ดูค่าบนสุดของ Stack โดยไม่เอาออก
- **isEmpty** – ตรวจสอบว่า Stack ว่างหรือไม่

การเขียนโปรแกรม การทำงานแบบ Stack

42

ตัวอย่างการทำงาน สมมติว่าเรามี Stack ว่างอยู่ แล้วทำการ Push ตัวเลขเข้าไปตามลำดับ:

Push(1) → Stack: [1]

Push(2) → Stack: [1, 2]

Push(3) → Stack: [1, 2, 3]

ตอนนี้ Stack มีค่า [1, 2, 3] โดยที่ 3 อยู่บนสุด

ถ้าเราทำ Pop():

Pop() → ได้ค่า 3 → Stack: [1, 2]

การเขียนโปรแกรม การทำงานแบบ Stack

43

การใช้งาน Stack ในชีวิตจริง

- การย้อนกลับ (Undo) ในโปรแกรมต่าง ๆ
- การจัดการวงเล็บในนิพจน์ทางคณิตศาสตร์
- การเรียกใช้ฟังก์ชันแบบซ้อน (Function Call Stack)
- การแปลงเลขฐาน หรือการเดินทางแบบย้อนกลับ (Backtracking)

ตัวอย่างการสร้างคลาส Stack

44

WS_oop >  Stack.py > ...

```
1  class Stack:
2      def __init__(self):
3          self.items = []
4
5      def is_empty(self):
6          return len(self.items) == 0
7
8      def push(self, item):
9          self.items.append(item)
10
11     def pop(self):
12         if not self.is_empty():
13             return self.items.pop()
14         return None
```

ตัวอย่างการสร้างคลาส Stack

45

```
15
16     def peek(self):
17         if not self.is_empty():
18             return self.items[-1]
19         return None
20
21     def size(self):
22         return len(self.items)
23
24     def __str__(self):
25         return f"Stack: {self.items}"
26
```

ตัวอย่างการเรียกใช้งานคลาส Stack

46

WS_oop > WS_stack.py > ...

```
1  from Stack import *
2
3  if __name__ == "__main__":
4      stack = Stack()
5      print("เพิ่มข้อมูลลงในสแต็ก:")
6      stack.push(10)
7      stack.push(20)
8      stack.push(30)
9      print(stack)
10     print("ดูค่าบนสุดของสแต็ก:", stack.peek())
11     print("นำข้อมูลออกจากสแต็ก:", stack.pop())
12     print(stack)
13     print("ขนาดของสแต็ก:", stack.size())
14     print("สแต็กว่างหรือไม่:", stack.is_empty())
```

เพิ่มข้อมูลลงในสแต็ก:
Stack: [10, 20, 30]
ดูค่าบนสุดของสแต็ก: 30
นำข้อมูลออกจากสแต็ก: 30
Stack: [10, 20]
ขนาดของสแต็ก: 2
สแต็กว่างหรือไม่: False

การเขียนโปรแกรม การทำงานแบบ Queue

47

Queue (คิว) เป็นโครงสร้างข้อมูล queuing ที่ทำงานตามหลักการ FIFO (First In, First Out) หรือ “เข้าแรกออกก่อน” หมายความว่า ข้อมูลที่ถูกเพิ่มเข้ามาก่อนจะถูกนำออกก่อนเสมอ

หลักการการทำงานของ Queue Queue มีการทำงานหลัก ๆ อยู่ 2 อย่าง:

1. Enqueue – การเพิ่มข้อมูลเข้าไปที่ท้ายคิว
2. Dequeue – การนำข้อมูลออกจากหัวคิว

นอกจากนี้ยังมีฟังก์ชันเสริมที่มักใช้ร่วมกัน เช่น:

- size () – ตรวจสอบขนาดของ Queue
- peek (หรือ front) – ดูค่าที่หัวคิวโดยไม่เอาออก
- isEmpty* – ตรวจสอบว่าคิวว่างหรือไม่

การเขียนโปรแกรม การทำงานแบบ Queue

48

ตัวอย่างการทำงาน

สมมติว่าเรามี Queue ว่างอยู่ แล้วทำการ Enqueue ตัวเลขเข้าไปตามลำดับ:

Enqueue(1) → Queue: [1]

Enqueue(2) → Queue: [1, 2]

Enqueue(3) → Queue: [1, 2, 3]

ตอนนี้ Queue มีค่า [1, 2, 3] โดยที่ 1 อยู่หัวคิว

ถ้าเราทำ Dequeue():

Dequeue() → ได้ค่า 1 → Queue: [2, 3]

การเขียนโปรแกรม การทำงานแบบ Queue

49

การใช้งาน Queue ในชีวิตจริง

- การต่อแถวซื้อของหรือรอรับบริการ
- การจัดการงานในระบบพิมพ์ (Print Queue)
- การส่งข้อมูลในระบบเครือข่าย
- การจำลองเหตุการณ์ (Simulation) เช่น การจราจร

ตัวอย่างการสร้างคลาส Queue

50

WS_oop >  Queue.py > ...

```
1  class Queue:
2      def __init__(self):
3          self.items = []
4
5      def is_empty(self):
6          return len(self.items) == 0
7
8      def enqueue(self, item):
9          self.items.append(item)  # เพิ่มที่ท้ายคิว
10
11     def dequeue(self):
12         if not self.is_empty():
13             return self.items.pop(0)  # เอาออกจากหัวคิว
14         else:
15             return None
```

ตัวอย่างการสร้างคลาส Queue

51

```
16
17     def peek(self):
18         if not self.is_empty():
19             return self.items[0]
20         else:
21             return None
22
23     def size(self):
24         return len(self.items)
25
```

ตัวอย่างการเรียกใช้คลาส Queue

52

WS_oop >  WS_Queue.py > ...

```
1  from Queue import *
2
3  if __name__ == "__main__":
4      q = Queue()
5      q.enqueue(10)
6      q.enqueue(20)
7      q.enqueue(30)
8
9      print("Dequeue:", q.dequeue()) # ได้ 10
10     print("Peek:", q.peak())       # เหลือ 20
11     print("Size:", q.size())       # เหลือ 2
```

Dequeue: 10
Peek: 20
Size: 2